

Data Structure

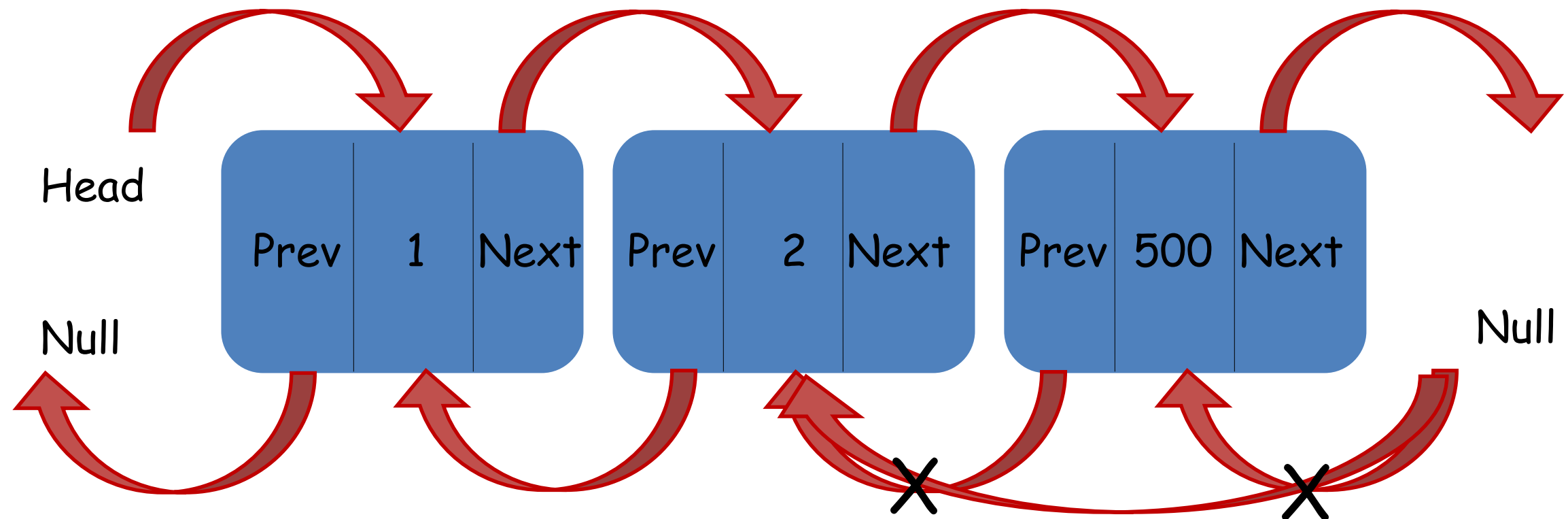
Level 1

Doubly Linked List Operation

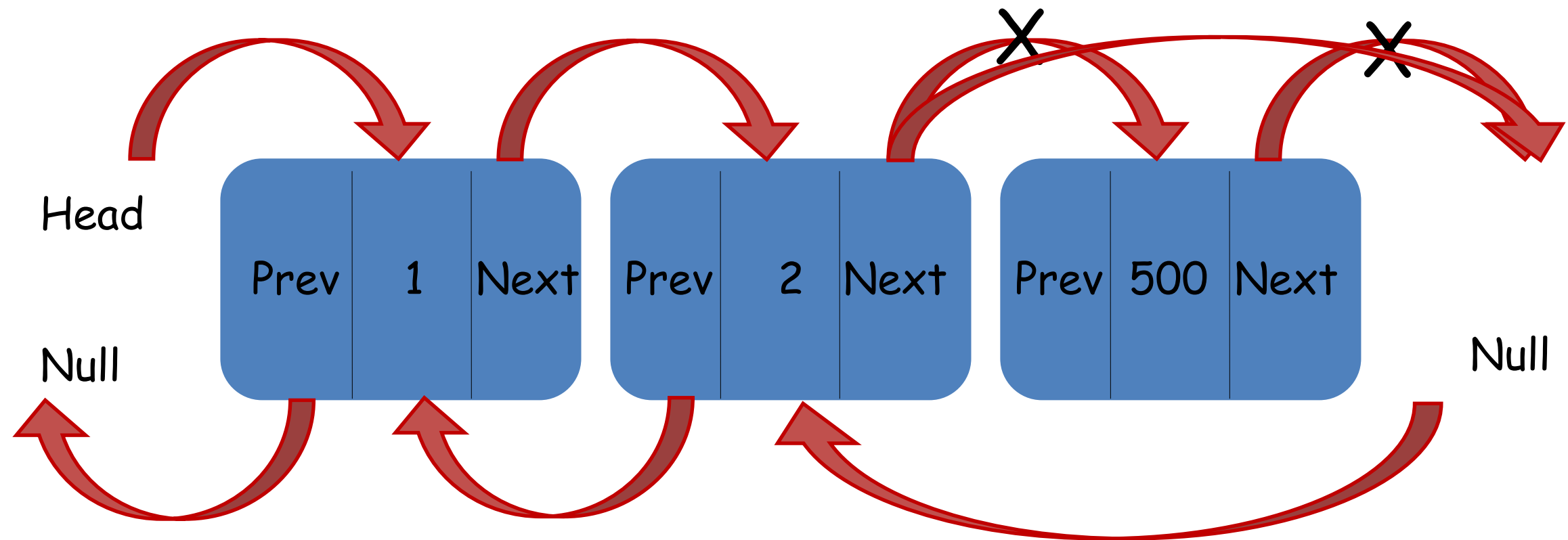
Data Structure

Level 1 **Delete Last Node**

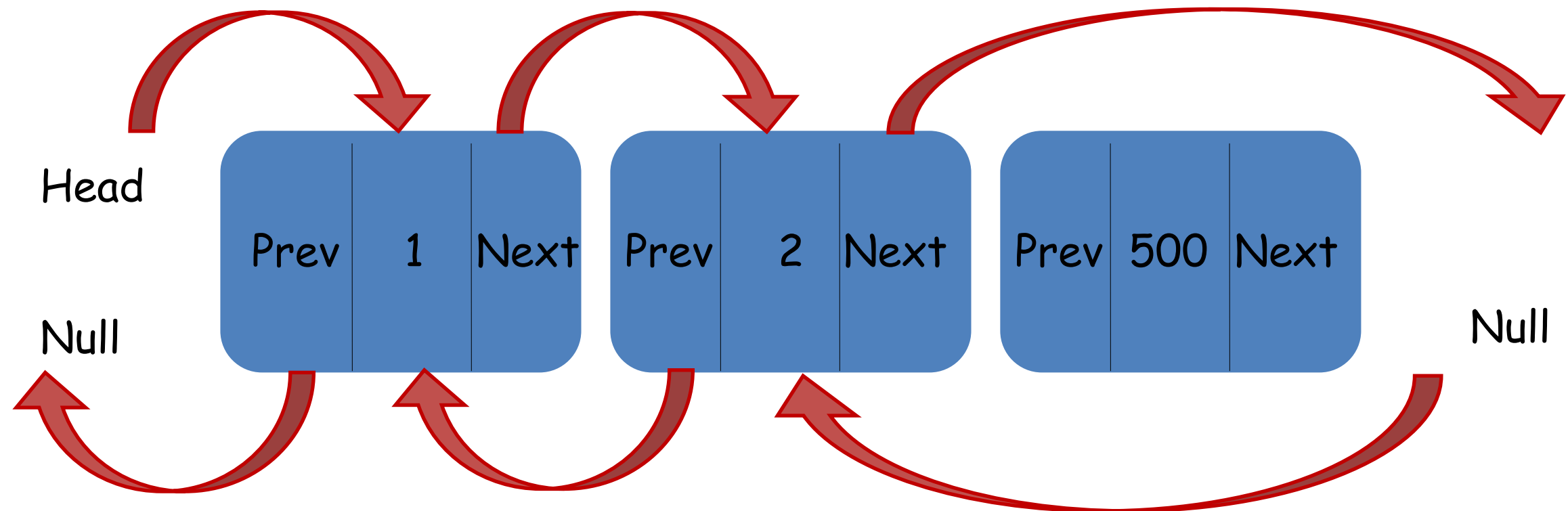
Doubley Linked List Data Structure



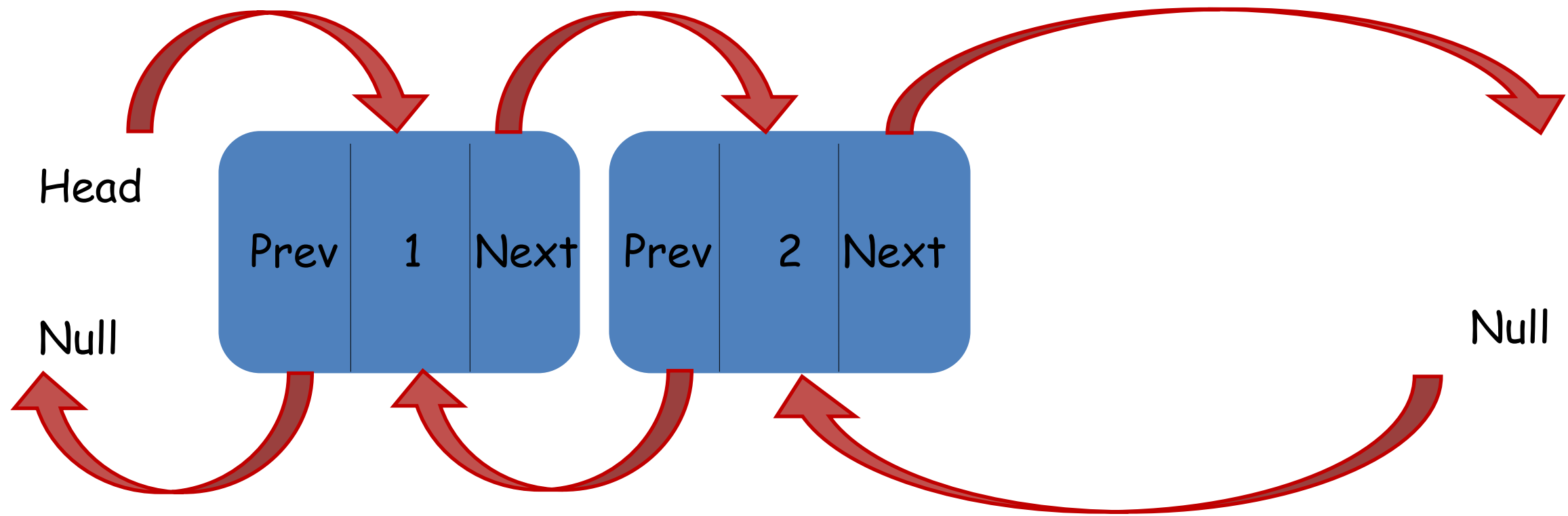
Doubley Linked List Data Structure



Doubley Linked List Data Structure



Doubley Linked List Data Structure



Definition

Removes the last node (tail) from a doubly linked list and updates the previous node correctly.

Explanation

- ◆ Deleting the last node requires:
 - ▣ Finding the last node
 - ▣ Setting the `next` pointer of the second-to-last node to `NULL`
 - ▣ Deleting the last node from memory
- ◆ Special handling is needed if the list has:
 - ▣ No nodes
 - ▣ Only one node

Procedures (Delete Last Node)

1. Check if List is Empty

Check:

If `head == NULL`

→ Result: Stop if the list is empty

2. Check if There is Only One Node

Check:

If `head->next == NULL`

Steps:

Delete `head`

Set `head = NULL`

→ Result: The list becomes empty

3. Find the Second-to-Last Node

Steps:

Start from `head`

Move while `current->next->next != NULL`

→ Result: `current` points to the second-to-last node

4. Store the Last Node

Step:

`Node\* temp = current->next`

→ Result: `temp` points to the node that will be deleted

Procedures (Delete Last Node)

5 Disconnect the Last Node

Step:

Set `current->next = NULL`

Result: The second-to-last node becomes the new last node

6 Delete the Last Node

Step:

`delete temp`

Result: The last node is removed from memory

💡 Example

Before:

NULL <--> 1 <--> 2 <--> 3 <--> 4 <--> 5 <--> NULL

Delete the last node:

Delete 5

After:

NULL <--> 1 <--> 2 <--> 3 <--> 4 <--> NULL

Note

- ◆ The code uses `current->next->next` to find the second-to-last node.
- ◆ Because the list is doubly linked, after deleting the last node, the new last node's `next` must be `NULL`.

Comparison

Delete First Node

- ✓ Time Complexity: $O(1)$
- ✓ No traversal needed
- ✓ Directly updates the head

Delete Last Node

- ✓ Time Complexity: $O(n)$
- ✓ Requires traversal to find the last node
- ✓ Updates the second-to-last node's `next` pointer

⊘ Warning

- ⚠ Always check `head == NULL` before accessing `head->next`.
- ⚠ If the list has only one node, `head` must be set to `NULL` after deletion.
- ⚠ In this implementation, the `prev` pointer of the new last node does not need to be changed because it already points to the correct previous node.

✦✦ Summary

- ✓ DeleteLastNode removes the last node from the list
- ✓ Requires traversal to find the second-to-last node
- ✓ Sets the new last node's next to NULL
- ✓ Handles empty, single-node, and multi-node lists

Thank you 😊